Reporte

Nombre:	**Matrícula**:
Valeria Ordaz Garcia Luis José Mendoza Serrano Omar Alejandro Salazar Castro	2169894 2223173 2226576
Nombre del curso: Componentes de lógica del negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad:	**Actividad**: Diseño arquitectural - ADRs
Fecha: 12/10/2026	

Bibliografia (Formato APA)

Documentos impartidos por el docente

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Objetivo:

Documentar las decisiones de arquitectura tomadas para EduTechAcademy en formato ADR (Architecture Decision Record): el contexto que las motivó, la decisión adoptada, las alternativas descartadas y sus consecuencias, con base en el BRD, las User Stories, el mapa de dominio y el Service Layer.

Procedimiento:

Registro de decisiones de arquitectura (ADR)

Enfoque

Un ADR registra una decisión arquitectónica significativa junto con su contexto y sus consecuencias, de modo que quien lea el proyecto después entienda por qué se diseñó así y no solo cómo (Nygard, 2011). Cada registro de este documento usa la misma estructura: **Estado**, **Contexto**, **Decisión**, **Alternativas consideradas** y **Consecuencias**.

Se documentan 11 decisiones. Todas ya están reflejadas en los puntos anteriores del diseño: la elección del patrón (Fowler, 2002), el mapa de dominio (Evans, 2003; Vernon, 2013) y el Service Layer. El ADR-001 es el que se anunció en la elección del patrón y el ADR-011 documenta la elección del Service Layer como capa de coordinación. Todos los registros tienen estado Aceptada; si una decisión cambia, se crea un ADR nuevo que la reemplaza y el anterior pasa a Reemplazada, sin borrarse.

Índice de decisiones

ID	Decisión	Reglas del BRD
ADR-001	Domain Model con Service Layer como patrón de lógica de negocio	RB-01 a RB-14, RES-01 a RES-07, PA-01, PA-02, PA-04
ADR-002	Aggregates pequeños relacionados por identificador; Inscripción como Aggregate central	RB-02, RB-03, RB-08, PA-02, PA-04
ADR-003	Umbrales de certificación dentro de Value Objects	RB-06, RB-07, RB-08, PA-02, PA-04
ADR-004	Historial de cambios de grupo y de instructor como entidades del dominio	RB-09, RB-11, RES-05, CE-04, PA-03
ADR-005	Permisos por rol en el Service Layer, fuera del dominio	RB-04, RB-05, RB-14, RES-04
ADR-006	Límites transaccionales y concurrencia optimista sobre Grupo	RB-02, RB-09, SUP-11
ADR-007	Domain Events publicados tras confirmar la transacción, con outbox y consistencia eventual	RB-10, SUP-14
ADR-008	Consultas y reportes de solo lectura directo a repositorios	REP-01 a REP-10, CE-03
ADR-009	Domain Service PoliticaInscripcion para los requisitos de inscripción	RB-01, RES-02, SUP-15
ADR-010	Precio fijado al inscribirse y estados explícitos como Value Objects	RB-03, RB-10, RES-06, SUP-13
ADR-011	Service Layer como capa de coordinación	RB-14, RES-04, US-01 a US-18, REP-01 a REP-10

ADR-001: Domain Model con Service Layer como patrón de lógica de negocio

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-01 a RB-14, RES-01 a RES-07, PA-01, PA-02, PA-04

Contexto

El BRD define 14 reglas de negocio y 7 restricciones que dependen unas de otras. La certificación, por ejemplo, depende de la calificación, la asistencia y el estado de la inscripción (RB-03, RB-06, RB-07 y RB-08). Los umbrales de 70/100 y 80 % intervienen en la consulta del estudiante (US-06), en la validación de Control Escolar (US-13) y en los reportes REP-06 y REP-07.

Además, varias invariantes deben cumplirse en la misma operación (cupo, pago, cambio de grupo) y el historial es un requisito. Los problemas actuales (PA-01, PA-02 y PA-04) vienen de que las reglas no están en un solo lugar.

Decisión

Se utilizará **Domain Model** para las reglas de negocio (cupo, requisitos, pago, criterios de certificación e historial), con un **Service Layer** encima que coordina los casos de uso. Los servicios solo orquestan; la decisión de negocio la toma el dominio. Las consultas de solo lectura quedan fuera del dominio (ver ADR-008). La elección del Service Layer como capa de coordinación se documenta en el ADR-011.

Alternativas consideradas

Alternativa	Por qué se descartó
Transaction Script	Puntuación 11 de 30 en la tabla comparativa de la sección Elección del patrón de lógica de negocio (ver documento de Elección del patrón). Cada caso de uso repetiría los umbrales y las validaciones, que es justo lo que hoy causa PA-02 y PA-04.
Table Module	Puntuación 17 de 30 en la misma tabla. Trabaja por conjuntos de registros por tabla; las reglas que cruzan Inscripción, Grupo y CambioDeGrupo no tienen un lugar claro y no hay objetos con identidad.

Consecuencias

- **A favor:** las reglas viven en un solo lugar y se pueden probar con objetos en memoria, sin base de datos.
- **A favor:** el lenguaje ubicuo (Inscripción, Cupo, Certificación) se refleja en clases y métodos.
- **A favor:** es coherente con Aggregates, Value Objects, Domain Events y Service Layer.
- **En contra / costo:** mayor curva de aprendizaje, más código y diseño inicial, y necesidad de mapear objetos a la base de datos. Se mitiga con un glosario de lenguaje ubicuo, aplicando el patrón solo donde hay reglas y usando repositorios por Aggregate.

ADR-002: Aggregates pequeños relacionados por identificador; Inscripción como Aggregate central

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-02, RB-03, RB-08, RB-09, PA-02, PA-04

Contexto

Curso, Grupo, Inscripción, Estudiante e Instructor se relacionan de forma constante. La regla de certificación (RB-08) necesita pago, asistencias, evaluaciones, cambios de grupo y certificación juntos para responder de forma consistente. Al mismo tiempo, un Aggregate grande aumentaría la contención y el costo de cargarlo.

Decisión

Se definen 7 Aggregates: Curso, Grupo, Instructor, Estudiante, Inscripción, SolicitudCapacitación y ClienteEmpresarial. Se relacionan solo por identificador (Inscripción guarda estudianteId y grupoId; Grupo guarda cursoId e instructorId; SolicitudCapacitación guarda clienteId y cursoId).

Inscripción agrupa Pago, Asistencia, Evaluación, CambioDeGrupo y Certificación. No es un Aggregate grande: hay una instancia por estudiante y grupo y sus colecciones están acotadas por la duración del curso. **SolicitudCapacitación** es un Aggregate propio para que ClienteEmpresarial no crezca sin límite. Control Escolar y Administración no son entidades: son actores.

Alternativas consideradas

Alternativa	Por qué se descartó
Aggregate grande por Estudiante o por Curso que contenga todo	Cargaría datos innecesarios en cada operación y haría que usuarios distintos compitieran por el mismo Aggregate.
Grupo que contenga sus Inscripciones	Todas las inscripciones modificarían el mismo Aggregate, aumentando los conflictos justo en las operaciones más concurridas (inscribir y registrar asistencia).
Pago, Asistencia y Evaluación como Aggregates separados	La certificación no tendría un lugar único donde verificar sus criterios de forma consistente (PA-02, PA-04).

Consecuencias

- **A favor:** cada Aggregate protege sus invariantes y la certificación se calcula con todos los datos de la inscripción juntos.
- **A favor:** los Aggregates pequeños se cargan y guardan rápido.
- **En contra / costo:** las operaciones que tocan dos Aggregates (inscribir, cambiar de grupo) requieren coordinación explícita, definida en el ADR-006.

ADR-003: Umbrales de certificación dentro de Value Objects

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-06, RB-07, RB-08, RB-12, RB-13, PA-02, PA-04

Contexto

La calificación mínima de 70/100 y la asistencia mínima de 80 % se usan en la consulta del estudiante, la validación de Control Escolar y dos reportes. Si cada consumidor repite el número, un cambio de política obliga a localizar y modificar varios lugares y es fácil que queden valores distintos.

Decisión

Los umbrales viven dentro de dos Value Objects inmutables: **Calificación** (valor de 0 a 100; aprueba() si el valor es mayor o igual a 70) y **PorcentajeAsistencia** (valor de 0 a 100; cumpleMinimo() si es mayor o igual a 80). La Inscripción los usa en evaluarCertificacion(); los servicios y reportes no repiten los números.

La calificación final es el promedio simple de las evaluaciones (SUP-09) y el porcentaje de asistencia es la proporción de sesiones registradas con presencia (SUP-10).

Alternativas consideradas

Alternativa	Por qué se descartó
Constantes repetidas en cada servicio o consulta	Es la causa de los errores de consolidación actuales y obliga a modificar varios lugares ante un cambio de política.
Parámetros de configuración leídos en cada consulta	El BRD no pide cambiarlos en ejecución; añadiría complejidad sin necesidad. Podría reconsiderarse si cambian por curso.

Consecuencias

- **A favor:** si la institución cambia el 70/100 o el 80 %, solo se modifica un Value Object.
- **A favor:** las reglas son fáciles de probar de forma aislada.
- **En contra / costo:** los umbrales quedan fijos en el código: cambiarlos implica una nueva versión del sistema.

ADR-004: Historial de cambios de grupo y de instructor como entidades del dominio

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-09, RB-11, RES-05, CE-04, REP-08, PA-03

Contexto

Hoy es difícil rastrear los cambios de grupo (PA-03). El BRD exige conservar el grupo anterior y el nuevo en cada cambio (RB-09) y el registro del instructor anterior cuando hay una sustitución (RB-11). Un historial claro y consultable es además un criterio de éxito (CE-04).

Decisión

El historial se modela como parte del dominio, no como una bitácora aparte. **CambioDeGrupo** es una entidad de Inscripción (grupo anterior, grupo nuevo, fecha, quién lo autorizó y motivo). **AsignaciónInstructor** es una entidad de Grupo con fecha de inicio y de fin (vacía si está vigente). Grupo garantiza que siempre exista un instructor vigente: sustituir cierra la asignación vigente y abre la nueva.

Alternativas consideradas

Alternativa	Por qué se descartó
Tablas de bitácora o auditoría manejadas a mano	El historial quedaría fuera del modelo y cada operación tendría que acordarse de escribirlo.
Sobrescribir el grupo o el instructor sin conservar el anterior	Incumple RB-09, RB-11 y RES-05.

Consecuencias

- **A favor:** la trazabilidad no se puede omitir porque es parte de la operación que cambia el estado.
- **A favor:** REP-08 se obtiene directamente del modelo.
- **En contra / costo:** hay más entidades y las colecciones crecen con el tiempo, aunque están acotadas por la duración del curso.

ADR-005: Permisos por rol en el Service Layer, fuera del dominio

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-04, RB-05, RB-14, RES-04, SUP-07

Contexto

RB-14 define permisos distintos para Estudiante, Instructor, Control Escolar, Administración y Cliente empresarial. RB-04 y RB-05 indican que el instructor es quien registra asistencia y evaluaciones. Control Escolar y Administración son actores del sistema, no conceptos con identidad dentro del dominio.

Decisión

El Service Layer verifica el rol antes de cada operación y el dominio solo protege reglas del negocio. Para el Instructor, el servicio comprueba con Grupo.esInstructor() que esté asignado al grupo; el

Estudiante solo accede a su propia información. Cada tabla de Application Services indica en la columna Rol quién puede ejecutar cada método.

Alternativas consideradas

Alternativa	Por qué se descartó
Verificar permisos dentro de los Aggregates	Los Aggregates dependerían de usuarios y roles, mezclando quién actúa con qué reglas de negocio se cumplen, y serían más difíciles de probar.

Consecuencias

- **A favor:** el dominio queda limpio y se prueba sin usuarios ni roles.
- **A favor:** los permisos se revisan en un solo lugar por caso de uso.
- **En contra / costo:** un servicio nuevo podría olvidar la verificación. Se mitiga listando el Rol de cada método y con pruebas de permisos por método.

ADR-006: Límites transaccionales y concurrencia optimista sobre Grupo

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-02, RB-09, RES-01, SUP-11

Contexto

Por regla general una operación debe modificar un solo Aggregate, pero inscribir ocupa un lugar en el Grupo y crea la Inscripción, y cambiar de grupo ocupa un lugar nuevo, libera el anterior y registra el CambioDeGrupo. Además, dos inscripciones simultáneas podrían tomar el mismo último lugar y rebasar la capacidad (RB-02). Sin un mecanismo de concurrencia, ambas transacciones podrían confirmarse y violar la invariante de capacidad máxima. Por eso se necesita control de concurrencia optimista sobre el Aggregate Grupo.

Decisión

La regla general es una operación por Aggregate. Las excepciones se limitan a los casos donde una invariante atraviesa dos Aggregates y debe cumplirse de forma atómica:

- **inscribir:** Grupo e Inscripción en una sola transacción.
- **cambiarGrupo y solicitarTransferencia:** Grupo anterior, Grupo nuevo e Inscripción en una sola transacción.
- **registrarAsistencia, registrarEvaluacion y validarCertificacion:** varias Inscripciones del grupo en una sola transacción de aplicación, trade-off aceptado porque el número está acotado por el cupo.

Grupo usa **control de concurrencia optimista** (número de versión) para que dos inscripciones no tomen el mismo último lugar. Una inscripción pendiente de pago ocupa lugar (SUP-11).

Alternativas consideradas

Alternativa	Por qué se descartó
Bloqueo pesimista del Grupo	Bloquea a todos los que se inscriben al mismo grupo aunque haya cupo de sobra.
Consistencia eventual para el cupo	Permitiría sobreventa de lugares y violaría RB-02.
Una transacción por inscripción al registrar asistencia	Una sesión podría quedar registrada solo para parte del grupo.

Consecuencias

- **A favor:** la capacidad máxima nunca se rebasa y los cambios de grupo no dejan cupos inconsistentes.
- **En contra / costo:** un conflicto de versión obliga a reintentar y el servicio lo informa al actor.
- **En contra / costo:** las transacciones que tocan varios Aggregates son una excepción que debe mantenerse documentada y no ampliarse.

ADR-007: Domain Events publicados tras confirmar la transacción, con outbox y consistencia eventual

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-03, RB-10, SUP-14, REP-02, REP-03, REP-05, REP-08

Contexto

Cancelar un grupo afecta a todas sus inscripciones y exige notificar, reembolsar o transferir (RB-10). Aceptar una solicitud de capacitación empresarial crea Estudiantes e Inscripciones (SUP-14). Hacer todo eso en la misma transacción que cambia el Grupo o la Solicitud tocaría muchos Aggregates. Además, un evento debe representar algo que realmente ocurrió. Si el servicio publica el evento antes de confirmar la transacción y luego la transacción falla, el evento anunciaría un hecho que no ocurrió. Y si el servicio se cae después de confirmar la transacción pero antes de publicar el evento, el evento se perdería. El patrón outbox garantiza que el evento se publique exactamente una vez, incluso ante fallos.

Decisión

El Aggregate registra el Domain Event cuando cambia de estado y el Service Layer lo publica **después** de confirmar la transacción, con entrega confiable mediante el patrón **outbox**. Dos flujos usan consistencia eventual:

- **GrupoCancelado:** Grupo.cancelar() se confirma en una transacción; como reacción, las inscripciones pasan a GrupoCancelado, se notifica a los estudiantes y se abre la opción de reembolso o transferencia.

- **CapacitaciónAceptada:** la solicitud se acepta en una transacción; como reacción, se crean los Estudiantes y las Inscripciones de los participantes.

Alternativas consideradas

Alternativa	Por qué se descartó
Publicar el evento antes de confirmar	Podría anunciar hechos que después no se guardaron.
Actualizar todas las inscripciones en la misma transacción que cancela el Grupo	Una transacción grande sobre muchos Aggregates, contraria al ADR-006.
Llamadas directas entre Aggregates	Los acopla entre sí y rompe los límites de cada uno.

Consecuencias

- **A favor:** notificaciones, reportes y reacciones quedan desacoplados del Aggregate que origina el hecho.
- **A favor:** ningún evento anuncia algo que no se completó.
- **En contra / costo:** durante un lapso el grupo ya está Cancelado pero algunas inscripciones aún no se actualizan.
- **En contra / costo:** quien consume los eventos debe tolerar que un mismo evento se entregue más de una vez.

ADR-008: Consultas y reportes de solo lectura directo a repositorios

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	REP-01 a REP-10, US-01, US-17, CE-03

Contexto

El catálogo de cursos y los reportes REP-01 a REP-10 no modifican estados ni aplican reglas complejas. Cargar Aggregates completos solo para armar una lista o un reporte sería innecesario.

Decisión

CatalogoCursosService y ReporteService consultan directamente los repositorios, sin cargar Aggregates, y devuelven DTO. Siguen verificando permisos (ADR-005). Los reportes de elegibilidad (REP-06 y REP-07) leen el EstadoCertificación que calculó el dominio y no vuelven a aplicar el 70/100 ni el 80 %.

Alternativas consideradas

Alternativa	Por qué se descartó
Armar los reportes cargando Aggregates completos	Costo innecesario para consultas que no cambian estado y riesgo de duplicar reglas fuera del dominio.

Consecuencias

- **A favor:** consultas más simples y rápidas.
- **A favor:** el dominio se mantiene enfocado en las operaciones que cambian estado.
- **En contra / costo:** si un reporte recalcula una regla en vez de leer el estado del dominio, aparecerían resultados distintos. Por eso la regla de leer el estado ya calculado es parte de esta decisión.

ADR-009: Domain Service PoliticaInscripcion para los requisitos de inscripción

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-01, RES-02, SUP-15

Contexto

RB-01 exige que el estudiante cumpla los requisitos del curso antes de inscribirse. La regla involucra a dos Aggregates, Estudiante y Curso, y ninguno debe contener al otro. Los requisitos son condiciones descritas en texto (SUP-15).

Decisión

La regla se modela como un **Domain Service sin estado**: PoliticaInscripcion.verificar(estudiante, curso). Comprueba los requisitos con los datos disponibles del estudiante o con la validación de Control Escolar y devuelve el resultado con el motivo si no se cumplen. InscripcionService.inscribir() lo invoca antes de crear la Inscripción.

Alternativas consideradas

Alternativa	Por qué se descartó
Método en Curso que reciba al Estudiante	Curso dependería del Aggregate Estudiante.
Validar en el Service Layer	Pondría lógica de negocio fuera del dominio, contra el ADR-001.

Consecuencias

- **A favor:** la regla tiene un solo lugar y no acopla a los Aggregates.
- **En contra / costo:** al ser texto, los requisitos no siempre se pueden comprobar automáticamente y pueden requerir validación de Control Escolar (SUP-15). Si se quisiera automatizar por completo, habría que estructurar los requisitos.

ADR-010: Precio fijado al inscribirse y estados explícitos como Value Objects

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026

Campo	Detalle
Reglas y problemas del BRD	RB-03, RB-10, RES-06, RES-07, SUP-13

Contexto

El precio del Curso puede cambiar con el tiempo (Curso.cambiarPrecio) y eso no debe afectar a quienes ya se inscribieron. Además, preguntas como si un grupo ya terminó, si un pago está confirmado o si un reembolso ya se realizó no deben depender de reglas implícitas.

Decisión

El monto del **Pago** copia el precio del Curso al momento de inscribirse y el Pago valida el monto recibido contra ese precio. Los estados son Value Objects explícitos: EstadoGrupo, EstadoInscripción, EstadoPago, EstadoCertificación y EstadoSolicitud. Un pago fallido o rechazado no cambia el estado (SUP-13) y el reembolso solo pasa a Reembolsado cuando se procesa. Este ADR complementa al ADR-003, que ya define los Value Objects Calificación y PorcentajeAsistencia. El ADR-010 extiende el uso de Value Objects a los estados del ciclo de vida (EstadoGrupo, EstadoInscripción, EstadoPago, EstadoCertificación y EstadoSolicitud).

Alternativas consideradas

Alternativa	Por qué se descartó
Que el Pago consulte el precio vigente del Curso	Un cambio de precio posterior alteraría inscripciones ya existentes.
Banderas booleanas sueltas (pagado, cancelado, finalizado)	Permiten combinaciones inválidas y esconden las reglas de transición entre estados.

Consecuencias

- **A favor:** las inscripciones existentes no se alteran por cambios de precio.
- **A favor:** las transiciones de estado son visibles y fáciles de probar.
- **En contra / costo:** hay que mantener la copia del precio y definir con claridad las transiciones válidas de cada estado.

ADR-011: Service Layer como capa de coordinación

Campo	Detalle
Estado	Aceptada
Fecha	12/10/2026
Reglas y problemas del BRD	RB-14, RES-04, US-01 a US-18, REP-01 a REP-10

Contexto

El BRD define 5 actores (Estudiante, Instructor, Control Escolar, Administración y Cliente empresarial) con permisos distintos (RB-14). Cada actor ejecuta casos de uso sobre los Aggregates del dominio. Si los actores llamaran directamente a los Aggregates, el dominio quedaría acoplado a la interfaz de usuario y a los permisos, violando el ADR-005. Además, varios casos de uso requieren coordinar más

de un Aggregate en una sola transacción (inscribir, cambiar de grupo), y los eventos de dominio deben publicarse después de confirmar los cambios (ADR-007). Se necesita una capa que reciba las peticiones de los actores, verifique permisos y coordine los casos de uso sin contaminar el dominio con lógica de aplicación.

Decisión

Se utiliza el patrón **Service Layer** (Fowler, 2002) con 8 Application Services: CatalogoCursosService, InscripcionService, AsistenciaService, EvaluacionService, CertificacionService, GrupoService, ReporteService y CapacitacionEmpresarialService. Cada Application Service agrupa los casos de uso de un mismo tema, verifica permisos (ADR-005), carga los Aggregates desde los repositorios, invoca los métodos del dominio en el orden correcto, controla la transacción según los límites del ADR-006, publica los Domain Events después de confirmar (ADR-007) y devuelve un DTO. El Service Layer no contiene reglas de negocio: solo coordina.

Alternativas consideradas

Alternativa	Por qué se descartó
Llamadas directas de los actores a los Aggregates	El dominio quedaría acoplado a la interfaz de usuario y a los permisos (RB-14), violando el ADR-005. Cada actor tendría que conocer los Aggregates y sus invariantes.
Patrón MVC (Model-View-Controller)	Mezcla la lógica de presentación con la lógica de aplicación. Los controladores terminarían conteniendo reglas de negocio, contra el ADR-001.
Patrón Hexagonal (Ports & Adapters)	Es más complejo de lo necesario para el alcance del proyecto. Aísla el dominio de la infraestructura, pero agrega puertos y adaptadores que no aportan valor en esta etapa.
Transaction Script en la capa de presentación	Concentra toda la lógica en procedimientos, que es justo lo que el ADR-001 busca evitar.

Consecuencias

- **A favor:** el dominio queda puro y testeable sin usuarios ni roles (ADR-005).
- **A favor:** los casos de uso se centralizan en servicios específicos y son fáciles de localizar.
- **A favor:** los permisos, las transacciones y la publicación de eventos se manejan en un solo lugar por caso de uso.
- **En contra / costo:** se agrega una capa más de indirección entre los actores y el dominio.
- **En contra / costo:** un servicio nuevo podría olvidar verificar permisos o publicar eventos. Se mitiga con la tabla de trazabilidad con las User Stories y con pruebas de permisos por método.
- **En contra / costo:** los servicios deben conocer los Aggregates y sus operaciones, lo que los acopla al modelo de dominio. Se mitiga manteniendo los servicios delgados y delegando toda decisión al dominio.